

CASE STUDY

Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Name: Karpaga Sundari V

Roll Number: 241401037

Experiment No.: _____

Date: _____

Course: _____

1. Aim

To study where low-code platforms work well and where they fall short for a large, complex enterprise system. The study uses one example case and suggests ways to reduce the problems found.

2. Procedure

This is a desk study. It uses a fictional case and general knowledge of low-code platforms.

Step 1. Choose an enterprise use case. Here it is loan processing for a large bank (a fictional scenario).

Step 2. List what the system needs: number of users, data size, rules, links to other systems, security and audit.

Step 3. Note what low-code platforms usually offer: forms, simple workflows, reports, ready connectors and role-based access.

Step 4. Compare each need with these strengths. Mark the fit as High, Medium or Low. These marks are the author's judgement.

Step 5. Find the limitations that appear for this use case.

Step 6. Suggest a way to reduce each limitation.

Step 7. Draw a hybrid design that uses low-code where it fits and normal code where it does not.

Step 8. Write the conclusion and list the facts to verify in the real platform's documentation.

3. Output

No screenshots were supplied for this report. The items below are **illustrative tables and diagrams** based on a fictional case. They are not measured results.

3.1 The example case: bank loan processing

A large bank wants one system for loan applications from all its branches and its mobile app. The team asks whether the whole system can be built on a low-code platform. This case is fictional.

Need	Detail in the example case
Users	Branch staff, loan officers, risk teams and customers using a mobile app.
Data and load	Many applications at the same time, with large records to store and search.
Rules	Detailed credit and risk rules with many conditions.
Links to other systems	Core banking system, credit bureau, document store and payment services.
Security and audit	Strict access control and a full record of every change.

Table 3.1: Needs of the example case (fictional).

3.2 Indicative fit of a low-code platform

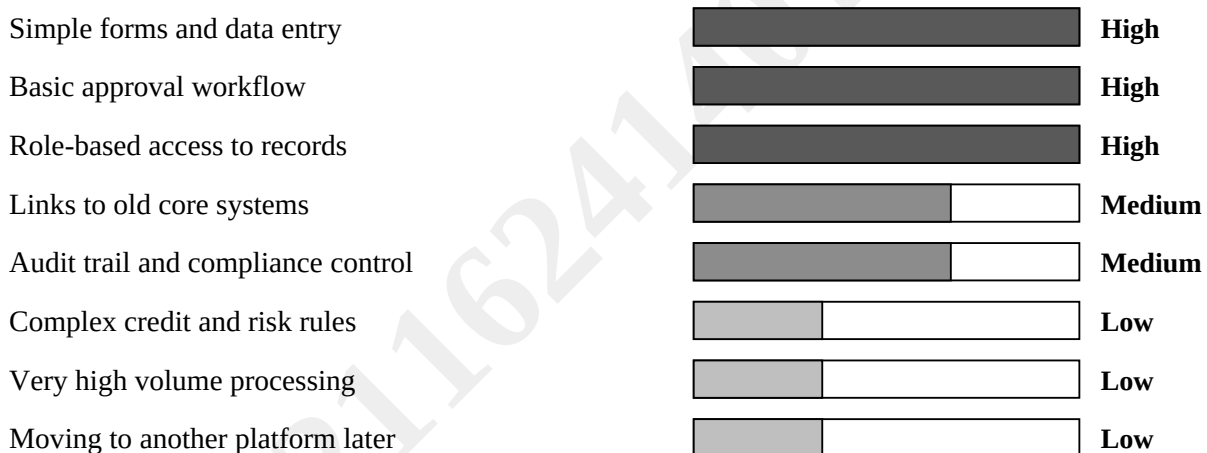


Figure 3.2: Indicative fit by need (the author's judgement for this example, not measured; real results depend on the platform).

3.3 Suggested hybrid design

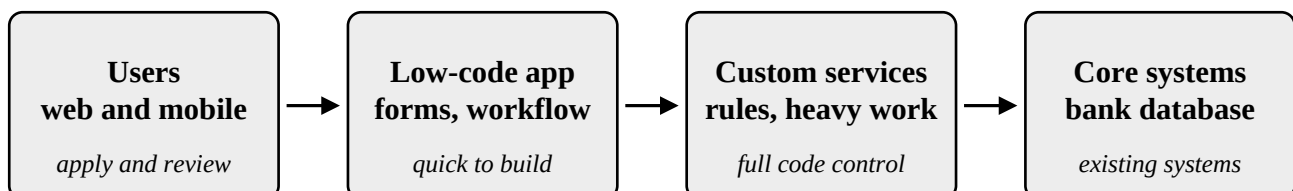


Figure 3.3: Low-code for the user-facing part, normal code for heavy logic (illustrative diagram).

3.4 Limitations found and ways to reduce them

Limitation	What it means in this case	Way to reduce it
Performance control	Many users at once may slow the app, and the team cannot tune the platform itself.	Test with realistic load early. Move heavy work to separate services.
Complex rules	Risk rules with many conditions are hard to build and read in visual tools or short scripts.	Keep the rules in a separate service or code module.
Integration limits	Old core banking systems may need custom connectors.	Use APIs and a middle layer. Confirm connector support first.
Security and compliance	The bank may need control over data location and detailed audit records.	Check the vendor's certificates and options. Ask the compliance team.
Vendor lock-in	App logic lives on the vendor's platform, so moving later is costly.	Document the data model and logic. Export data regularly.
Testing and versions	There are fewer tools for automated tests and code review.	Use separate environments, written test plans and release checklists.
Cost at scale	Pricing may grow as users and usage grow.	Estimate the cost for the full user base before choosing.
Governance	Easy building by many staff can create apps that nobody manages.	Keep an owner list and an approval step for new apps.

Table 3.4: Limitations of low-code for the example case (general points; verify for your platform).

3.5 Choosing between low-code and normal code

Low-code is a good choice when	Normal code is a better choice when
The app is mostly forms, reports and simple workflows.	The system has heavy rules and calculations.
Speed of delivery matters most.	Very high volume or fast response is required.
The user base is small or medium.	Many other systems must be linked in detail.
The rules may change often and staff must edit them.	Strict control over data, testing and hosting is needed.

Table 3.5: Simple guide for choosing an approach (general guidance).

4. Final Result

The case study showed that low-code platforms suit forms, simple workflows and fast delivery. They have limits for a large enterprise system with heavy rules, high volume, many links to other systems and strict compliance. A hybrid design is suggested for the example case. The findings come from a fictional scenario and general study, not from tests or measurements.